LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6: INHERITANCE (PEWARISAN)

Dibuat Oleh :

Nuevalen Refitra Alswando 1030724300008

PROGRAM STUDI INFORMATIKA

FAKULTAS INFORMATIKA

UNIVERSITAS TELKOM SURABAYA

2026

1. TUJUAN PRAKTIKUM

1. Mengerti dan memahami inheritance secara mendalam
2. Memahami konsep inheritance dan polymorfisme serta dapat mengimplementasikannya dalam suatu program Java

2. LANDASAN TEORI

2.1 Pengertian Inheritance

Inheritance (pewarisan) adalah suatu cara untuk menurunkan suatu kelas yang lebih umum menjadi suatu kelas yang lebih spesifik. Inheritance merupakan salah satu ciri utama bahasa pemrograman berorientasi objek. Java mendukung inheritance dengan memungkinkan satu kelas untuk bersatu dengan kelas lainnya ke dalam deklarasinya.

Istilah Penting:

- **Superclass/Base Class**: Kelas yang menurunkan (kelas dasar)
- **Subclass/Derived Class**: Kelas yang diturunkan (kelas turunan)
- **Karakteristik superclass akan dimiliki juga oleh subclass-nya**

2.2 Konsep Hierarki Kelas

Semakin ke bawah dalam hierarki, kelas akan semakin bersifat spesifik. Contoh:

- **Manusia** adalah superclass dari **Pria** dan **Wanita**
- Pria dan Wanita mewarisi state dan behaviour kelas Manusia

2.3 Polymorphism

Polymorphism adalah kemampuan sebuah Object untuk mengambil beberapa bentuk yang berbeda agar tidak terjadi duplikasi Object. Konsep ini memungkinkan digunakannya suatu interface yang sama untuk memerintah objek agar melakukan aksi yang secara prinsip sama namun secara proses berbeda.

2.4 Keyword Penting

Keyword	Fungsi
extends	Membuat subclass mewarisi parent class
super	Mengakses member superclass dari subclass

3. IMPLEMENTASI DAN PEMBAHASAN

3.1 Contoh Dasar Inheritance

Langkah Kerja:

1. Buka NetBeans

2. Buat Project baru: **Modul6_Inheritance**
3. Buat class-class sesuai kode berikut

File: ClassA.java

```java
package modul06.inheritance;

public class ClassA {
    int x;
    int y;

    void tampilXY() {
        System.out.println("nilai x: " + x + " nilai y: " + y);
    }
}
```

Penjelasan:

- ClassA adalah **superclass** yang memiliki dua atribut (x dan y) dan satu method (tampilXY())
- Method tampilXY() menampilkan nilai atribut x dan y

File: ClassB.java

```java
package modul06.inheritance;

public class ClassB extends ClassA {
    int z;

    void jumlahXY() { System.out.println("jumlah: " + (x + y + z)); }
}
```

Penjelasan:

- ClassB adalah **subclass** yang mewarisi ClassA menggunakan keyword extends
- ClassB memiliki atribut tambahan z yang tidak dimiliki superclass
- ClassB dapat mengakses atribut x dan y dari superclass karena inheritance
- Method jumlahXY() menjumlahkan ketiga atribut: x + y + z

File: DemoInheritance.java

```java
package modul06.inheritance;

public class DemoInheritance {
    public static void main(String[] args) {
        // Membuat objek superclass
        ClassA superclass = new ClassA();

        // Membuat objek subclass
        ClassB subclass = new ClassB();

        System.out.println("=== SUPERCLASS ===");
        superclass.x = 3;
        superclass.y = 4;
        superclass.tampilXY();

        System.out.println("\n=== SUBCLASS ===");
        // Member superclass dapat diakses dari subclass-nya
        subclass.x = 1;
        subclass.y = 2;
        subclass.tampilXY();

        // Member tambahan hanya ada di subclass
        subclass.z = 5;
        subclass.jumlahXY();
    }
}
```

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=58141" -Dfile.encoding=UTF-8 -Ds
=== SUPERCLASS ===
nilai x: 3 nilai y: 4

=== SUBCLASS ===
nilai x: 1 nilai y: 2
jumlah: 8

Process finished with exit code 0
```

Penjelasan:

1. Objek superclass dibuat dari ClassA dan objek subclass dibuat dari ClassB
2. Pada superclass: atribut x dan y diisi, lalu method tampilXY() dipanggil
3. Pada subclass: atribut x dan y dari superclass dapat diakses dan dimodifikasi
4. Subclass memiliki atribut tambahan z yang tidak dimiliki superclass
5. Method jumlahXY() pada subclass dapat mengakses x, y (dari parent) dan z (milik sendiri)

Analisis Output:

Baris	Penjelasan
nilai x: 3 nilai y: 4	Output dari method tampilXY() pada objek superclass
nilai x: 1 nilai y: 2	Output dari method tampilXY() yang diwarisi subclass
jumlah: 8	Hasil perhitungan x(1) + y(2) + z(5) = 8

3.2 Keyword super

File: Employee.java

```java
package modul06.KeywordSuper;

public class Employee {
    private String name;
    String departemen;  // default access (package-private)

    // Constructor
    public Employee(String s) {
        name = s;
    }

    // Method untuk menampilkan nama
    public void tampilNama() {
        System.out.println("nama: " + name);
    }
}
```

Penjelasan:

- Class Employee adalah superclass dengan atribut name (private) dan departemen (default)
- Constructor menginisialisasi atribut name
- Method tampilNama() menampilkan nilai name

File: Manager.java

```java
public class Manager extends Employee {
    private String alamat;

    // Constructor dengan memanggil constructor parent
    public Manager(String nama, String s) {
        /* Memanggil constructor Employee */
        super(nama);
        alamat = s;
    }

    public void tampilAlamat() {
        /* Menginisialisasi variabel departemen yang ada pada superclass */
        super.departemen = "Personalia";

        /* Memanggil method tampilNama() yang ada pada superclass */
        super.tampilNama();

        /* Menampilkan variabel departemen yang telah diinisialisasi */
        System.out.println("alamat: " + alamat);
        System.out.println("departemen: " + super.departemen);
    }

    public static void main(String[] args) {
        /* Membuat objek Manager */
        Manager adi = new Manager( nama: "adi",  s: "sukabirus");
        adi.tampilAlamat();
    }
}
```

Penjelasan Tiga Penggunaan Keyword super:

super(parameter) - Memanggil Constructor Parent

- Harus ditulis di baris pertama dalam constructor subclass
- Digunakan untuk menginisialisasi atribut yang diwarisi dari parent class

super.namaAtribut - Mengakses Atribut Parent

- Mengakses atribut departemen milik superclass dari dalam subclass

- Hanya bisa mengakses atribut yang tidak private

super.namaMethod() - Memanggil Method Parent

- Memanggil method tampilNama() yang didefinisikan di superclass
- Berguna ketika method tersebut di-override di subclass

3.3 Overloading Method

File: Lagu.java

```java
package modul06.OverloadingMethod;

public class Lagu {
    String pencipta;
    String judul;

    // Method IsiParam dengan 1 parameter (Overloading #1)
    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    // Method IsiParam dengan 2 parameter (Overloading #2)
    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }

    // Method untuk menampilkan informasi
    void CetakKeLayar() {
        System.out.println("Judul: " + judul + ", pencipta: " + pencipta);
    }
}
```

Penjelasan:

- Class Lagu memiliki dua method dengan nama yang sama: IsiParam()
- Overloading #1: IsiParam(String param1) → menerima 1 parameter, atribut pencipta diisi nilai default
- Overloading #2: IsiParam(String param1, String param2) → menerima 2 parameter, kedua atribut diisi sesuai input

File: DemoOver2.java

```java
package modul06.OverloadingMethod;

public class DemoOver2 {
    public static void main(String[] args) {
        // Membuat dua objek Lagu
        Lagu d, e;
        d = new Lagu();
        e = new Lagu();

        // Memanggil IsiParam dengan 1 parameter (menggunakan overloading #1)
        d.IsiParam( param1: "Lagu 1");

        // Memanggil IsiParam dengan 2 parameter (menggunakan overloading #2)
        e.IsiParam("kepastian yang kutunggu", "GiGi");

        // Menampilkan hasil
        d.CetakKeLayar();
        e.CetakKeLayar();
    }
}
```

Penjelasan:

1. Dua objek Lagu dibuat: d dan e

2. Objek d memanggil IsiParam("Lagu 1") → Java otomatis memilih method dengan **1 parameter**
3. Objek e memanggil IsiParam("kepastian yang kutunggu", "GiGi") → Java otomatis memilih method dengan **2 parameter**

3.4 Overriding Method

File: ClassA_Override.java

```java
package modul06.OverridingMethod;

public class ClassA {
    public void tampilkanKeLayar() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

Penjelasan:

- ClassA_Override adalah superclass dengan method tampilkanKeLayar()

File: ClassB_Override.java

```java
package modul06.OverridingMethod;

public class ClassB extends ClassA {
    @Override
    public void tampilkanKeLayar() {
        // Memanggil method milik superclass menggunakan keyword super
        super.tampilkanKeLayar();

        // Menambahkan implementasi khusus di subclass
        System.out.println("Method milik class B dipanggil...");
    }
}
```

Penjelasan:

- ClassB_Override meng-override method tampilkanKeLayar() dengan implementasi yang lebih spesifik
- Annotation @Override digunakan untuk menandakan bahwa method ini meng-override parent class
- Keyword super.tampilkanKeLayar() digunakan untuk memanggil versi method dari superclass

File: DemoInheritance_Override.java

```java
package modul06.OverridingMethod;

public class DemoInheritance {
    public static void main(String[] args) {
        // Membuat objek dari subclass ClassB
        ClassB subOb = new ClassB();

        // Memanggil method yang sudah di-override
        // Yang dieksekusi adalah method milik ClassB, bukan ClassA
        subOb.tampilkanKeLayar();
    }
}
```

```
Run  modul06.OverridingMethod.DemoInheritance

"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Method milik class A dipanggil...
Method milik class B dipanggil...

Process finished with exit code 0
```

Penjelasan:

- Objek subOb dibuat dari class ClassB_Override (subclass)
- Saat method tampilkanKeLayar() dipanggil, Java menggunakan mekanisme **runtime polymorphism**
- Method yang dieksekusi adalah versi dari ClassB, bukan ClassA

Analisis Output:

Baris Output	Sumber Method	Penjelasan
Method milik class A dipanggil...	super.tampilkanKeLayar() di ClassB	Method parent dipanggil secara eksplisit via keyword super
Method milik class B dipanggil...	Method override di ClassB	Implementasi tambahan dari subclass

---

4. ANALISIS DAN PEMBAHASAN

4.1 Perbedaan Overloading vs Overriding

Aspek	Overloading	Overriding
Lokasi	Dalam satu class yang sama	Parent ↔ Child class
Signature	Harus berbeda (parameter)	Harus sama persis
Return type	Bebas	Harus sama/kovarian
Access modifier	Bebas	Tidak boleh lebih restriktif
Polymorphism	Compile-time (Static)	Runtime (Dynamic)
Keyword	Tidak ada khusus	@Override, super

4.2 Aturan Penggunaan Keyword super

1. **Posisi**: super(parameter) harus di baris **pertama** dalam constructor
2. **Akses Atribut**: Tidak bisa mengakses atribut private parent via super
3. **Overriding**: super.method() berguna untuk memanggil versi parent dari method yang di-override
4. **Hanya di Subclass**: Keyword super hanya bisa digunakan di dalam kelas turunan

4.3 Keuntungan Inheritance

1. **Reusability**: Kode dapat digunakan kembali tanpa menulis ulang
2. **Extensibility**: Mudah menambah fungsionalitas baru
3. **Maintainability**: Perubahan pada parent class otomatis berlaku di child class
4. **Organization**: Mengelompokkan kelas dalam hierarki yang logis

5. KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. **Inheritance** memungkinkan pembuatan kelas baru (child/subclass) yang mewarisi atribut dan method dari kelas yang sudah ada (parent/superclass) menggunakan keyword extends.
2. **Java hanya mendukung single inheritance** (satu kelas hanya bisa punya satu parent langsung).
3. **Keyword super** digunakan untuk:
    - Memanggil constructor parent class
    - Mengakses method parent yang di-override
    - Mengakses field parent yang di-hide
4. **Overloading Method** memungkinkan pembuatan dua atau lebih method dengan nama yang sama dalam satu kelas, tetapi dengan jumlah atau tipe data parameter yang berbeda (Compile-time Polymorphism).
5. **Overriding Method** memungkinkan child class memberikan implementasi spesifik untuk method yang sudah ada di parent class, dengan syarat signature method sama persis (Runtime Polymorphism).
6. **Polymorphism** adalah kemampuan sebuah Object untuk mengambil beberapa bentuk yang berbeda, memungkinkan digunakannya interface yang sama untuk memerintah objek melakukan aksi yang secara prinsip sama namun secara proses berbeda.
7. Pemahaman yang baik tentang inheritance dan polymorphism sangat penting untuk menciptakan desain sistem yang terstruktur, maintainable, dan scalable dalam pemrograman berorientasi objek.

8. DAFTAR PUSTAKA

1. Modul Praktikum Pemrograman Berorientasi Objek Modul 6: Inheritance
2. Deitel, P.J. & Deitel, H.M. (2017). Java How to Program. Pearson Education.
3. Oracle. (2024). The Java™ Tutorials: Inheritance. https://docs.oracle.com/javase/tutorial/java/IandI/